

SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery

Complete every task in Postman using the JSONPlaceholder API. Read each instruction carefully — later tasks build on what you did in earlier ones.

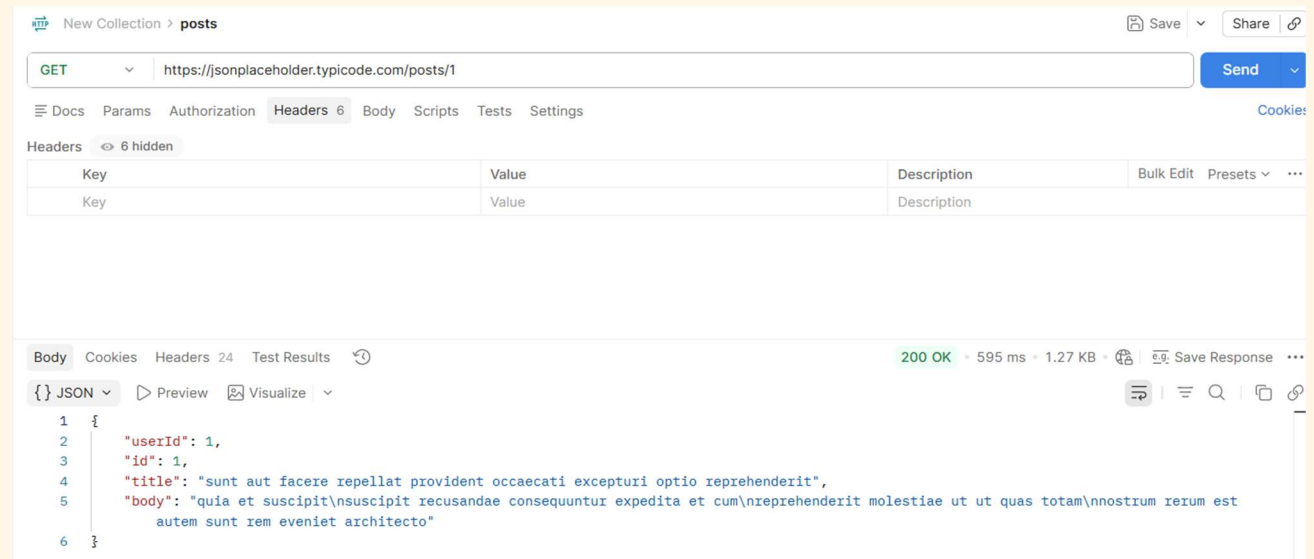
Before you start: you only need a browser and Postman. Base URL for every task: <https://jsonplaceholder.typicode.com> — no signup or API key required.

Tasks

TASK 1 Reading Data Without Headers

a) Send a GET request to `/posts/1` — leave the Headers and Body tabs completely empty.

Ans:



b) Record the status code you get back.

Ans: 200 OK

c) In one sentence, explain why this request works even with nothing in the Headers tab.

Ans: This request works because GET requests retrieve public data and do not require a request body or special headers.

TASK 2 Filtering With Query Parameters

a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.

Ans:

GET

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Query Params

Key	Value	Description
☒ postId	1	

Body Cookies Headers 24 Test Results 200 OK • 827 ms • 40.78 KB

{ } JSON Preview Visualize

```

1  [
2    {
3      "postId": 1,
4      "id": 1,
5      "name": "id labore ex et quam laborum",
6      "email": "Eliseo@gardner.biz",
7      "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreic
      nam sapiente accusantium"
8    },
9    {
10     "postId": 1,
11     "id": 2,
12     "name": "quo vero reiciendis velit similique earum",
13     "email": "Jayne_Kuhic@sydney.com",
14     "body": "est natus enim nihil est dolore omnis voluptatem numquam\nnet omnis occaecati quod ullam at\ nvoluptatem error expedita
      pariatur\nnihil sint nostrum voluptatem reiciendis et"
15    },
16    {
17     "postId": 1,
18     "id": 3,
19     "name": "odio adipisci rerum aut animi",
20     "email": "Nikita@garfield.biz",
21     "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet laudantium\nomnis quibusdam del
      quia accusamus maiores nam est\ncum et ducimus et vero voluptates excepturi deleniti ratione"
22    },
23  ]
  
```

b) Confirm the URL automatically updates to include ?postId=1.

Ans: Yes, after adding the query parameter in the Params tab, Postman automatically updates the URL to include ?postId=1

GET Send

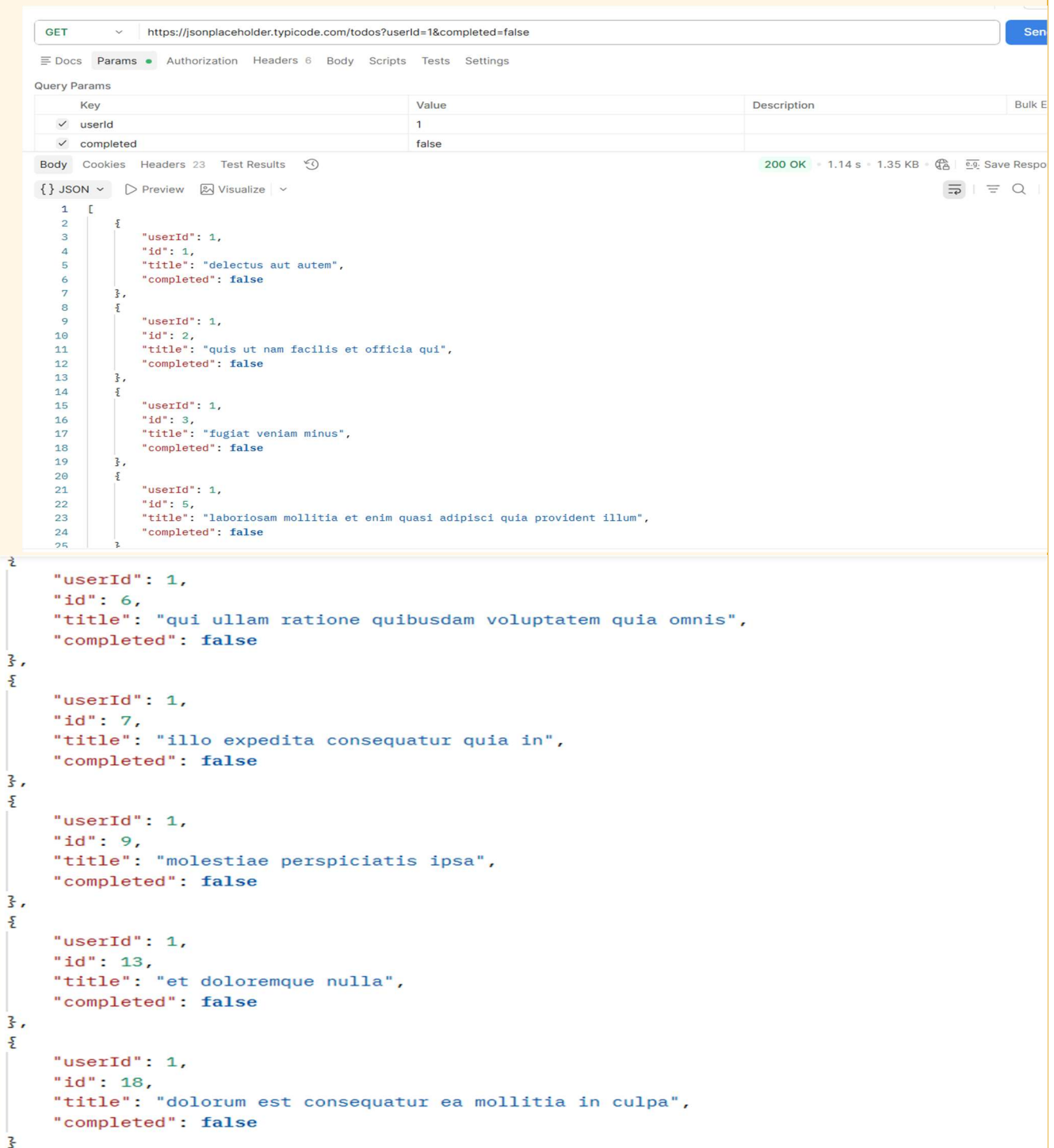
Docs Params Authorization Headers 6 Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit
☒ postId	1		

c) Now combine two filters at once on /todos: userId = 1 and completed = false.

Ans:



The screenshot shows a Postman interface with a GET request to `https://jsonplaceholder.typicode.com/todos?userId=1&completed=false`. The Params tab is active, showing two query parameters: `userId` with value `1` and `completed` with value `false`. The Body tab shows the response in JSON format, which is an array of 5 todo items for user 1 that are not completed.

```

[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  },
  {
    "userId": 1,
    "id": 5,
    "title": "laboriosam mollitia et enim quasi adipisci quia provident illum",
    "completed": false
  },
  {
    "userId": 1,
    "id": 6,
    "title": "qui ullam ratione quibusdam voluptatem quia omnis",
    "completed": false
  },
  {
    "userId": 1,
    "id": 7,
    "title": "illo expedita consequatur quia in",
    "completed": false
  },
  {
    "userId": 1,
    "id": 9,
    "title": "molestiae perspiciatis ipsa",
    "completed": false
  },
  {
    "userId": 1,
    "id": 13,
    "title": "et doloremque nulla",
    "completed": false
  },
  {
    "userId": 1,
    "id": 18,
    "title": "dolorum est consequatur ea mollitia in culpa",
    "completed": false
  }
]

```

d) In one sentence, explain what changed in the results compared to not using any params.

Ans: Using query parameters filters the results, so only records matching the specified conditions are returned instead of the complete list.

TASK 3 Sending Data With Headers & a Body

a) Send a POST request to /posts.

Ans:

GET https://jsonplaceholder.typicode.com/posts Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 24 Test Results 200 OK 685 ms 7.98 KB Save Response ...

{ } JSON Preview Visualize ...

```

1  [
2    {
3      "userId": 1,
4      "id": 1,
5      "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6      "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7    },
8    {
9      "userId": 1,
10     "id": 2,
11     "title": "qui est esse",
12     "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13    },
14    {
15     "userId": 1,
16     "id": 3,
17     "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18     "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
19    }
20  ]

```

b) In the Headers tab, add: Content-Type: application/json

Ans:

GET https://jsonplaceholder.typicode.com/posts Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Headers 6 hidden

Key	Value	Description	Bulk Edit	Presets
☒ Content-Type	application/json			

Body Cookies Headers 24 Test Results 200 OK 445 ms 7.98 KB Save Response

{ } JSON Preview Visualize ...

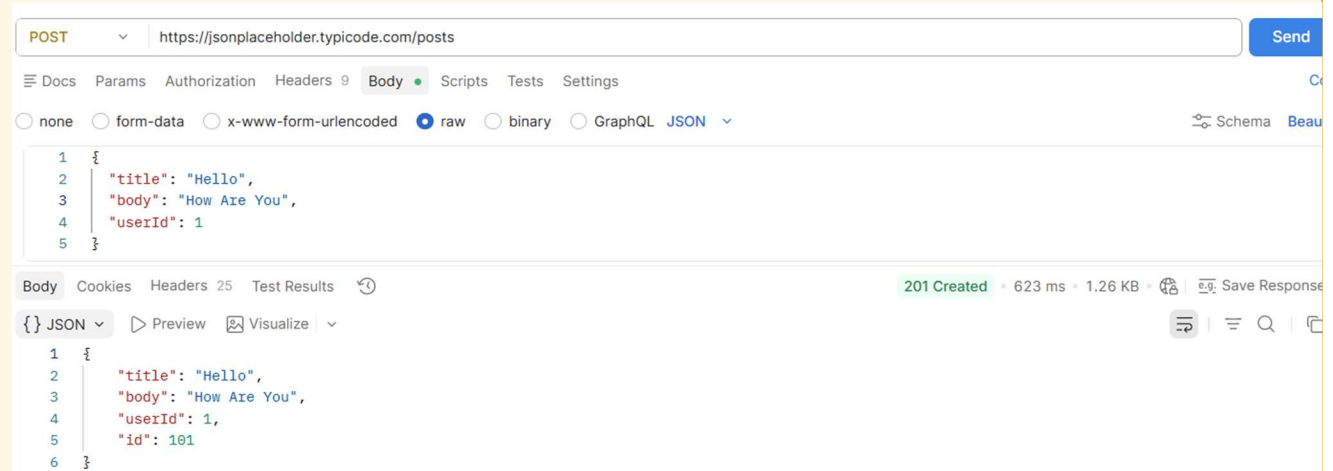
```

1  [
2    {
3      "userId": 1,
4      "id": 1,
5      "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6      "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7    },
8    {
9      "userId": 1,
10     "id": 2,
11     "title": "qui est esse",
12     "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13    },
14    {
15     "userId": 1,
16     "id": 3,
17     "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18     "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
19    }
20  ]

```

c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.

Ans:



d) Record the status code and the new id the server assigned.

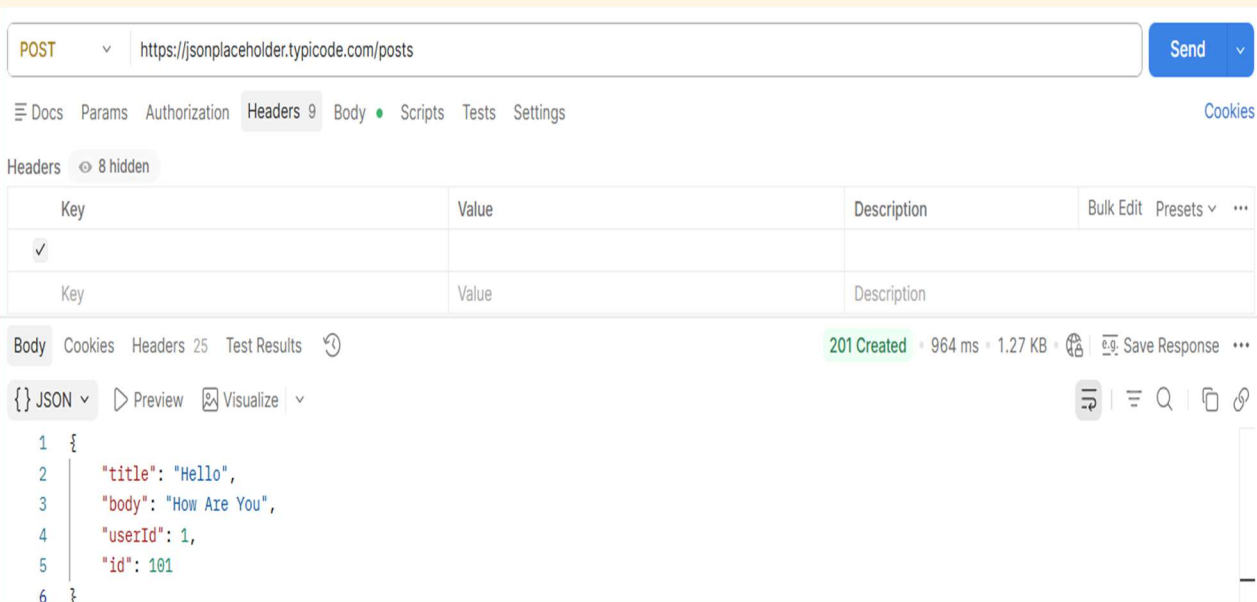
Ans: Status Code: 201 (Created)

Id: 101

TASK 4 Headers vs. No Headers

a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.

Ans:



b) Compare the response to Task 3 — is it different? Record what you observe.

Ans: The response is almost the same as Task 3. JSONPlaceholder accepts the request even without the Content-Type header.

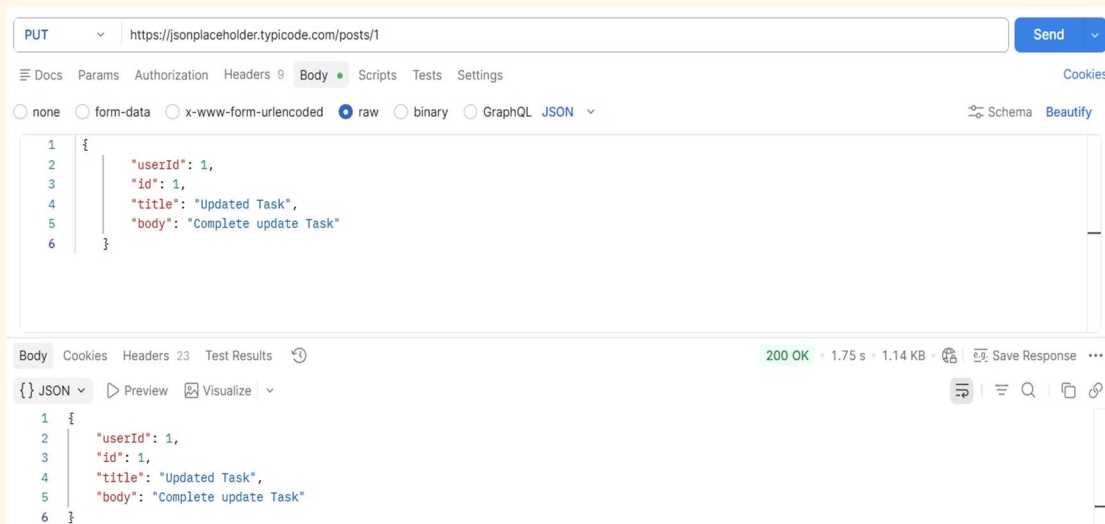
c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

Ans: Headers tell the server what type of data is being sent in the request. The Content-Type header helps the server correctly understand the JSON body. JSONPlaceholder is a practice API, so it accepts the request without this header, but many real APIs would reject the request or return an error.

TASK 5 PUT vs. PATCH

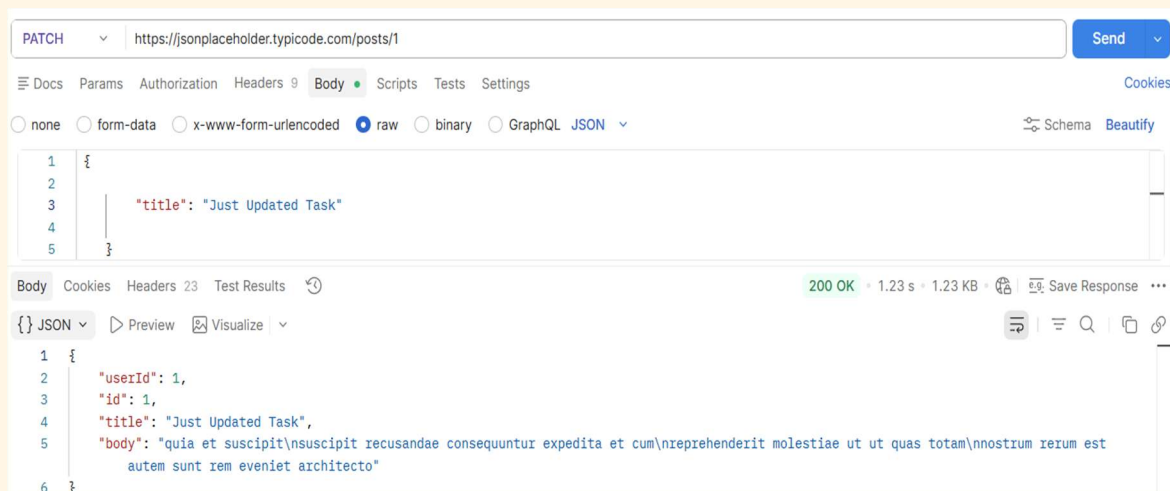
a) Send a PUT request to `/posts/1` that replaces the entire post (include id, title, body, and userId in the body).

Ans:



b) Send a PATCH request to the same `/posts/1` that changes only the title.

Ans:



c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

Ans: PUT replaces the entire resource, while PATCH updates only the specified field.

Final Challenge — A Realistic Request Flow

Concepts used: *this task combines query params, headers (with and without), and the request body — everything from Tasks 1 through 4 — in one flow.*

TASK 6 The Mini Feedback System

Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

Ans:

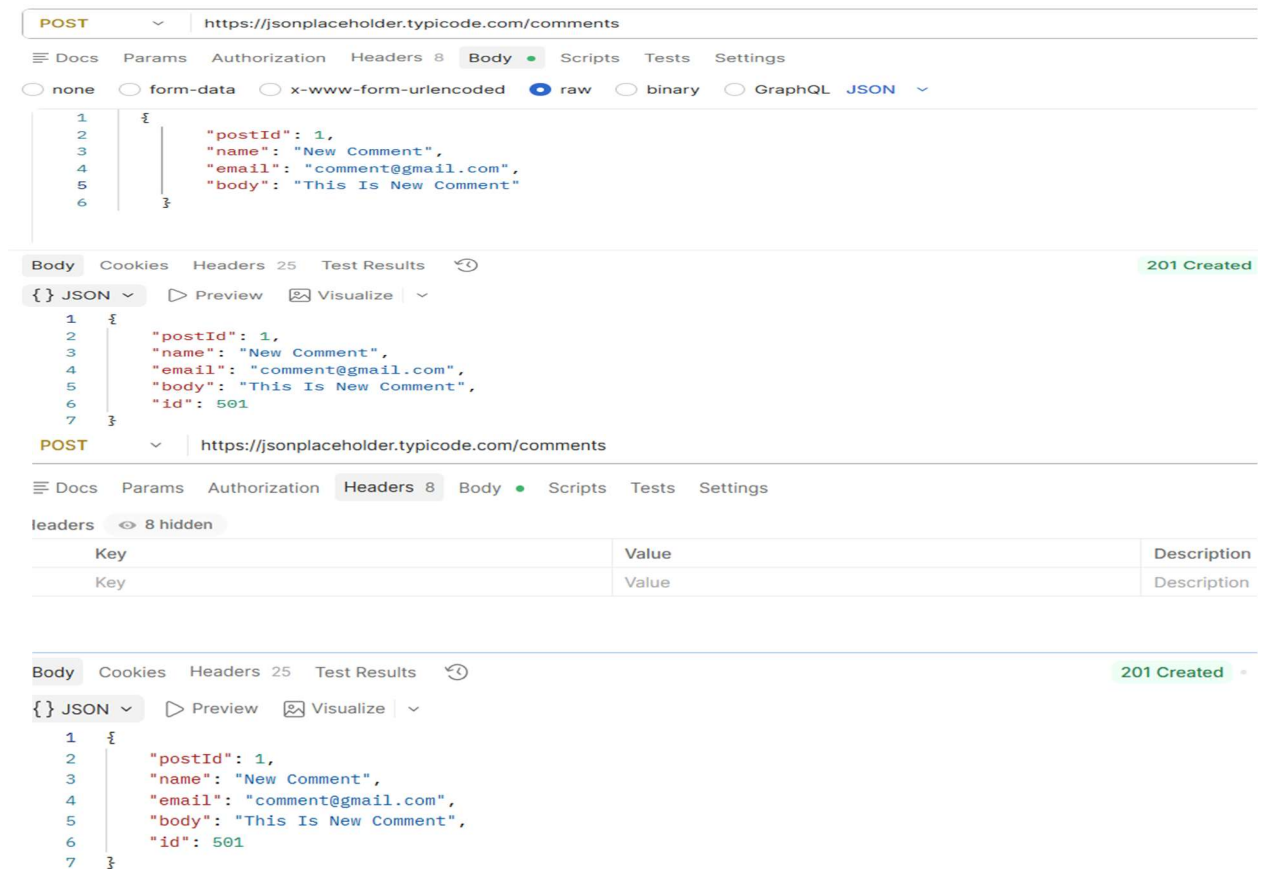
The screenshot shows a Postman interface with a GET request to `https://jsonplaceholder.typicode.com/comments?postId=1`. The request is sent, and the response is a 200 OK status with a response time of 1.11 s and a size of 40.78 KB. The response body is a JSON array of 3 comments.

```

1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14    "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nnihil sint nostrum voluptatem reiciendis et"
15  },
16  {
17    "postId": 1,
18    "id": 3,
19    "name": "odio adipisci rerum aut animi",
20    "email": "Nikita@garfield.biz",
21    "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet laudantium\nomnis quibusdam delectus saepe quia accusamus maiores nam est\nlaborum et dictum et ut autem repellat et repellendus"
22  }
23 ]
  
```


- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.

Ans:



POST `https://jsonplaceholder.typicode.com/comments`

Docs Params Authorization Headers 8 Body • Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "postId": 1,
3   "name": "New Comment",
4   "email": "comment@gmail.com",
5   "body": "This Is New Comment"
6 }

```

Body Cookies Headers 25 Test Results 201 Created

{ } JSON Preview Visualize

```

1 {
2   "postId": 1,
3   "name": "New Comment",
4   "email": "comment@gmail.com",
5   "body": "This Is New Comment",
6   "id": 501
7 }

```

POST `https://jsonplaceholder.typicode.com/comments`

Docs Params Authorization Headers 8 Body • Scripts Tests Settings

headers 8 hidden

Key	Value	Description
Key	Value	Description

Body Cookies Headers 25 Test Results 201 Created

{ } JSON Preview Visualize

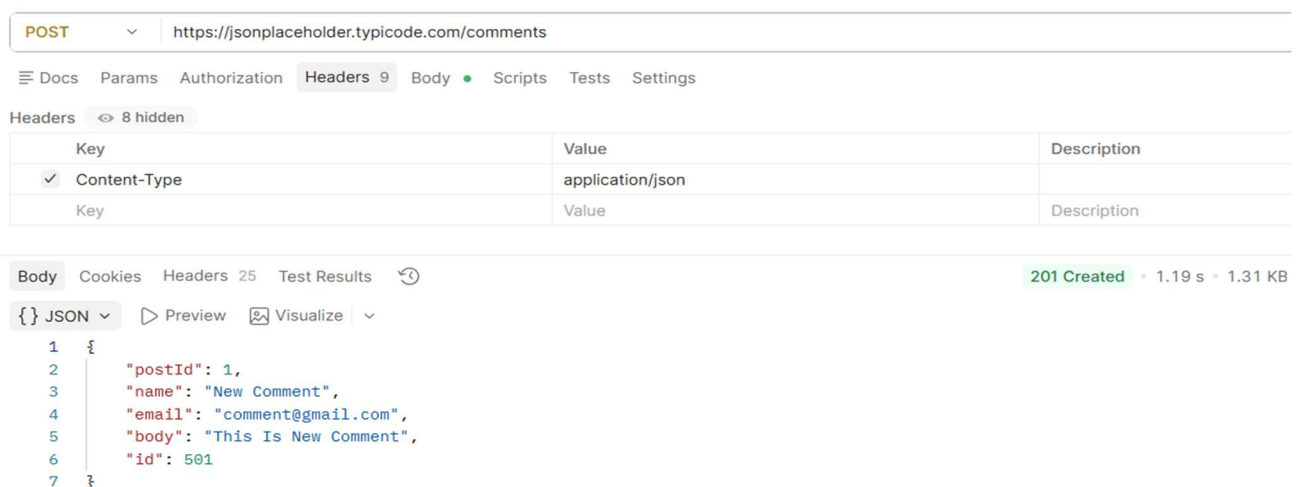
```

1 {
2   "postId": 1,
3   "name": "New Comment",
4   "email": "comment@gmail.com",
5   "body": "This Is New Comment",
6   "id": 501
7 }

```

- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.

Ans:



POST `https://jsonplaceholder.typicode.com/comments`

Docs Params Authorization Headers 9 Body • Scripts Tests Settings

Headers 8 hidden

Key	Value	Description
✓ Content-Type	application/json	
Key	Value	Description

Body Cookies Headers 25 Test Results 201 Created • 1.19 s • 1.31 KB

{ } JSON Preview Visualize

```

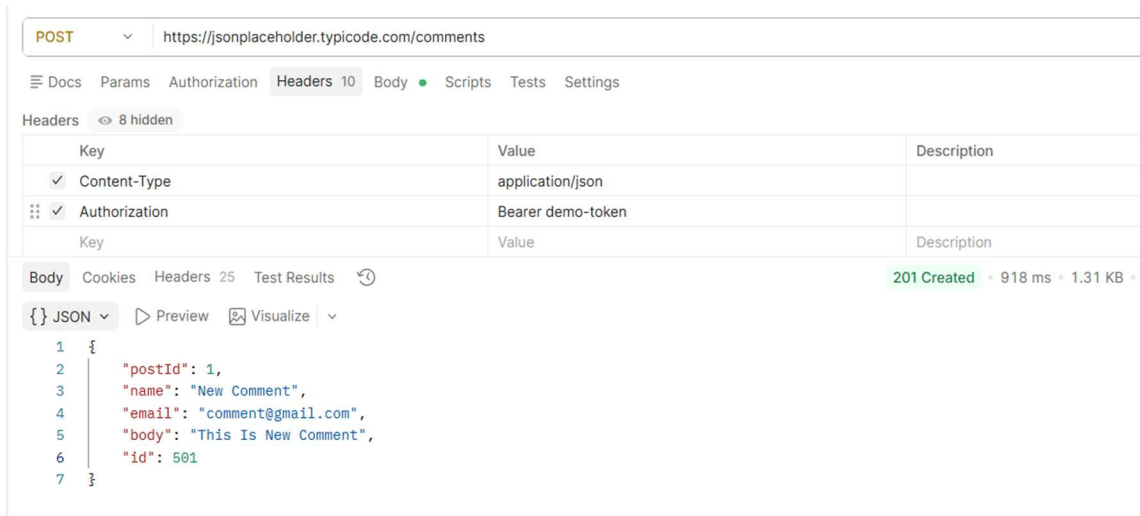
1 {
2   "postId": 1,
3   "name": "New Comment",
4   "email": "comment@gmail.com",
5   "body": "This Is New Comment",
6   "id": 501
7 }

```

Both requests return the same response because JSONPlaceholder accepts requests even without the Content-Type header.

- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.

Ans:



- e) Record the status code for every step above (a–d) in a short table or list.

Ans:

Step	Request	Status Code
a	GET /comments?postId=1	200 OK
b	POST /comments (No Header)	201 Created
c	POST /comments (Content-Type)	201 Created
d	POST /comments (Content-Type + Authorization)	201 Created

- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

Ans: In a real API, the server would verify the Authorization token before accepting the request. If the token was missing or invalid, the server would return an error such as 401 Unauthorized or 403 Forbidden. Only authenticated users would be allowed to submit feedback.